

Dynamic Programming II

Workshop – Term 2, Week 2

Yoshi He, Matthew Gu, Austin Song

Outline

1 Housekeeping

2 Revision of DP

- What is DP
- 1D DP
- Recursion vs Iteration
- 2D+ DP

3 Touring the classics

- The Knapsack Problem
- Longest Increasing Subsequence
- Longest Common Subsequence

4 On graphs??

- DP on trees
- DP on DAGs

5 Appendix and resources

Attendance!

Please scan the attendance form so you can get free food



VJudge For Workshop

If you're confident with the content for today, feel free to get started on problems!

Link: <https://vjudge.net/contest/821604>

Outline

1 Housekeeping

2 Revision of DP

- What is DP
- 1D DP
- Recursion vs Iteration
- 2D+ DP

3 Touring the classics

- The Knapsack Problem
- Longest Increasing Subsequence
- Longest Common Subsequence

4 On graphs??

- DP on trees
- DP on DAGs

5 Appendix and resources

What is DP?

Is a technique used to efficiently solve bigger problems(or states) by breaking it into similar smaller ones, forming connections between and states and storing values.

1 Optimal Substructure:

- smaller versions of whole problem can be combined to make the bigger version.
- (there is a recurrence)

2 Overlapping subproblems:

- If we didn't store these values, we would need to compute them a bunch of times.

Basic example

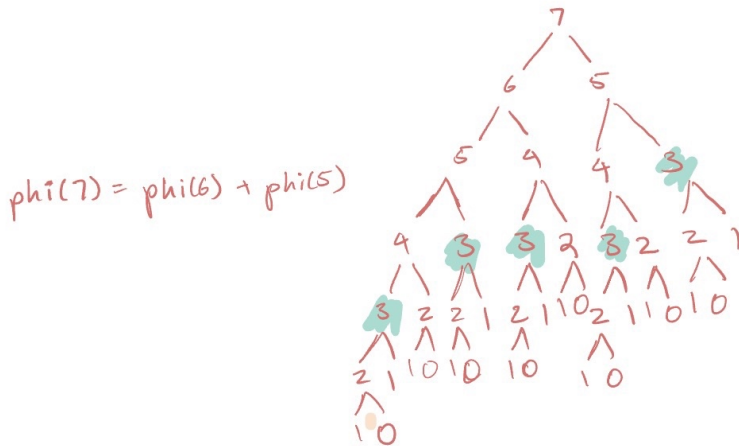


Figure: Fibonacci Sequence

Components of DP

1 State/subproblem

- what does $dp(i\dots)$ represent?
- includes all the dimensions we need to store

2 Recurrence

- How do the subproblems relate to each other?
- how does knowing the value of some sub-problems help us figure out others?

3 base cases

- What cases can we not figure out using the recurrence?
- things we need to define for our recurrence to have enough to work off of.

4 Order of Computation

- Important for bottom up/tabulation.
- Ensure that the dependencies of each subproblem is computed by the time we compute it.

5 output/answer

1D DP

Mario wants star

Mario is playing scuffed Mario party, he sees a star 67 spaces ahead, he wonders how many different ways there are to land exactly on the star.

On every turn he rolls a 6 sided die, 2 ways are different if the sequence of dice rolls differ at any point. You can assume the spaces on the board do not contain anything that affects Mario's movement.

Cool Diagram

Star 67 spaces ahead!



Solution

Let's say we are interested in knowing the number of ways to get to the 67th space.

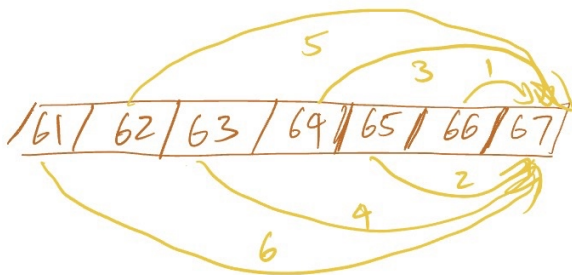
If we know how many ways there are to get to the 66th, 65th ... and 61th space we can figure this out quite easily!

The only way to get to space 67 is through one of these 6 previous spaces.

The last move in each sequence consists of moving from exactly one of these spaces to space 67. (no double counting) The number of ways to get to space 67 is the sum of the number of ways to get to each of these spaces $>.<$

This doesn't apply only to the 67th space, but every space :3.

Very Nice Diagram



Solution

1 State: Let space i be the space i steps in front of Mario's current position.

$dp(i) :=$ number of ways to get to space i .

2 Transition:

$$dp(i) := \sum_{j=1}^6 dp(i-j)$$

3 Base Cases:

- $dp(0) = 1$
either
- $dp(i) = 0$ for $i < 0$. or
- special case for $dp(i), i \in \{1...5\}$

4 Answer: $dp(67)$.

5 time complexity $O(67 \times 6) = O(1)$. (fake).

Recursion vs iteration

1 Bottom up/iteration:

- start from $dp(1)$ and fill upwards
- all dependencies of $dp(i)$ are computed when we try to figure out its value.

2 Top down:

- immediately ask what is $dp(67)$ and figure things out that we don't already know.
-

2D

Mario meets Annoying fish

Now let's say that some spaces contains coins (space i contains $\text{coins}[i]$ coins, $\text{coin}[i] \geq 0$ for all i). If Mario steps on a space that contains coins he picks all of them up and adds them to his inventory.

There is now also an annoying fish on space 40 that won't let you pass unless you pay it 23 money. This includes moving past space 40 without landing on it.

Now how many ways does can arrive at the 67th space containing the star?

Visual Representation of Annoying fish



Changes

- Now we need to keep track of how much money we have.
- Changes to the recurrence:

$$dp(i, c) := \sum_{j=1}^6 dp(i - j, c - \text{coins}[i])$$

- We would also need to check if we get stopped by the annoying fish.

$$dp(i, c) := 0$$

If $i \geq 40$ and $c - \text{coins}[i] < 23$.

- time complexity: $O(67 * 23 * 6)$

Note: We are not limited to 2 dimensions. For example if Mario is only interested in ways to land on the star in 20 moves. (what would we do now)?

State Transition

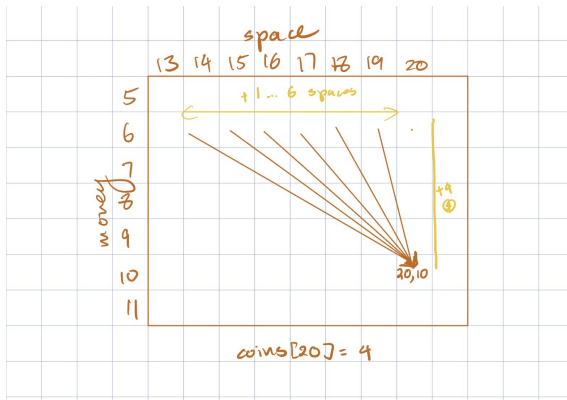


Figure: States

Outline

1 Housekeeping

2 Revision of DP

- What is DP
- 1D DP
- Recursion vs Iteration
- 2D+ DP

3 Touring the classics

- The Knapsack Problem
- Longest Increasing Subsequence
- Longest Common Subsequence

4 On graphs??

- DP on trees
- DP on DAGs

5 Appendix and resources

Problem

Piddy's Guest List

D. Piddy is making his guest list for another one of his world-renowned DP parties. There are N potential guests; guest i has a **freak score** f_i and a **cost** c_i (catering, security, NDAs – guests aren't cheap).

His budget is W , and each guest is invited at most once. Maximise the total freakiness (sum of freak scores) of the guest list.

Constraints:

- $N \leq 10^3$, $W \leq 10^5$.

Samples

Sample Cases

Sample Input: (first line is N and W ; each later line is c_i f_i)

4 10

2 3

3 4

4 5

5 6

Sample Output:

13

(Invite guests 1, 2, and 4: cost $2 + 3 + 5 = 10 \leq W$, freak score $3 + 4 + 6 = 13$.)

Solution

What do we need in our state?

- Consider the guests one at a time – each is a binary choice: **invite** or **don't**.
- After deciding the first i guests, all that matters going forward is the budget left.
- So, this motivates:

1 **State:** $dp(i, c) := \text{max freak score using the first } i \text{ guests with budget } c.$

2 **Transition:**

$$dp(i, c) = \max \left(\underbrace{dp(i-1, c)}_{\text{skip}}, \underbrace{dp(i-1, c - c_i) + f_i}_{\text{invite}} \right)$$

(the “invite” option only exists when $c_i \leq c$).

3 **Base Cases:** $dp(0, c) = 0$ for all c – no guests, no freaks.

- Our answer is simply $dp(N, W)$.
- Our time complexity is $O(NW)$.

Problem

Piddy's Problems II

Every D. Piddy DP party needs entertainment, so he's lining up some problems for his guests to solve. To keep the energy building, he wants the difficulties to come out **nondecreasing** – a smooth ramp keeps the party going.

He has N candidate problems written down in some order, with problem i rated at difficulty d_i . Keeping this order, find the length of the longest nondecreasing subsequence of difficulties.

Constraints:

- $N \leq 5 \times 10^3$.

Samples

Sample Cases

Sample Input: (number of problems, then difficulties in listed order)

8

3 1 4 1 5 9 2 6

Sample Output:

4

(One longest nondecreasing run of difficulties is 1, 4, 5, 9 – or 1, 4, 5, 6.)

Solution

What do we need in our state?

- Every valid run *ends* on some problem – let's index our subproblem by that final problem.
- To extend to problem i , glue d_i onto the best earlier run whose last difficulty is $\leq d_i$.
- So, this motivates:

1 **State:** $dp(i) :=$ length of the longest nondecreasing run **ending at** index i .

2 **Transition:**

$$dp(i) = 1 + \max_{\substack{j < i \\ d_j \leq d_i}} dp(j)$$

where the max over an empty set is taken to be 0.

3 **Base Cases:** $dp(i) = 1$ when no valid j exists (the problem stands alone).

- Our answer is $\max_i dp(i)$ – the best run might end on *any* problem.
- Our time complexity is $O(N^2)$.

P.S. There's a slicker $O(N \log N)$ approach – ask us about it after the workshop!

Problem

Who's Gonna Match My Freak?

D. Piddy lines up his male guests in one row and his female guests in another, each guest labelled by their **freak score**. He wants to pair guests across the two rows, matching equal freak scores, with **no two pairings crossing**.

Given the two rows A (length N) and B (length M), find the largest number of non-crossing matched pairs.

Constraints:

- $N, M \leq 3 \times 10^3$.

Samples

Sample Cases

Sample Input: (N and M , then row A , then row B ; numbers are freak scores)

```
7 5
1 2 3 2 4 1 2
2 4 3 1 2
```

Sample Output:

```
4
```

(One set of 4 non-crossing matches is 2, 3, 1, 2 – or 2, 4, 1, 2.)

Solution

What do we need in our state?

- Scan both rows from the front; all that matters is how far we are in each, i.e. the prefixes $A[1..i]$ and $B[1..j]$.
- Compare the last guests A_i and B_j : if their freak scores match, pair them and shrink both rows; otherwise drop one and retry.
- So, this motivates:

1 State: $dp(i, j) :=$ most matches between prefixes $A[1..i]$ and $B[1..j]$.

2 Transition:

$$dp(i, j) = \begin{cases} 1 + dp(i - 1, j - 1) & \text{if } A_i = B_j \\ \max(dp(i - 1, j), dp(i, j - 1)) & \text{if } A_i \neq B_j \end{cases}$$

3 Base Cases: $dp(0, j) = dp(i, 0) = 0$ – an empty row matches nobody.

- Our answer is simply $dp(N, M)$.
- Our time complexity is $O(NM)$.

Outline

1 Housekeeping

2 Revision of DP

- What is DP
- 1D DP
- Recursion vs Iteration
- 2D+ DP

3 Touring the classics

- The Knapsack Problem
- Longest Increasing Subsequence
- Longest Common Subsequence

4 On graphs??

- DP on trees
- DP on DAGs

5 Appendix and resources

Reexamining DP's Conditions

What were our conditions for using DP?

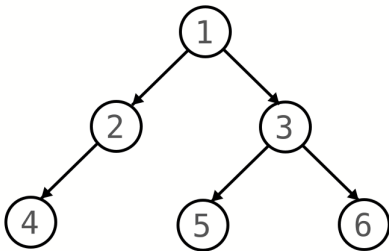
- No **cyclic** dependencies: we don't need $dp(x)$ to compute $dp(x)$.
 - For instance, $dp(x) = dp(x - 1) + 1$, $dp(0) = 1$ is good.
 - $dp(x) = dp((x - 1) \bmod n) + dp((x + 1) \bmod n)$ is terrible.

- Uh that's pretty much the important one.

Graphical Representation

We can draw our transitions in a visual way!

- Create a **node** for each state, and a **directed edge** for each transition.



An edge from $u \rightarrow v$ means that $dp(u)$ fetches its value off $dp(v)$.

Question: What special property does this graph have?

Trees

Trees are a type of graph with no cycles!

Arborescences

< arbor, arboris [f.]

An **arborescence** is just a rooted tree such that all edges are directed and point out of the root down to the leaves.

We'll just refer to them as 'trees' from now on though.

Theoretically, if DP only requires the lack of cycles, maybe we can DP on these weird funny directed trees!

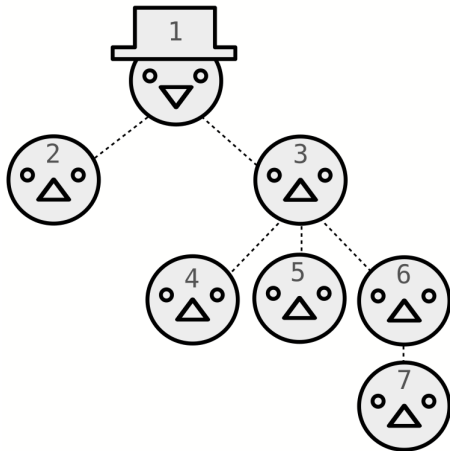
Subordinates

Subordinates

You run a company with N workers. You are worker 1, the general director. For all other workers $i \in \{2, \dots, N\}$, their boss is p_i .

Determine for each employee (including yourself) the number of people under their command.

Subordinates – Diagram



Subordinates – Solution

Let's say we store the graph via an adjacency list, where edges run from p_v to v .

- We could compute the number of 'reachable' nodes under a particular v in $O(N)$...
- Running a new DFS/BFS at each employee yields an $O(N^2)$ solution.

How can we do better?

- Let's define a **state**: $dp(v) :=$ number of subordinates for employee v .
- If we knew all other dp values, how could we compute $dp(v)$?
 - It's just 1 plus the sum of dp for all of v 's neighbours (alt. *children*)!
 - So our **transition** is simply

$$dp(v) = 1 + \sum_{i \in \text{adj}(v)} dp(i).$$

- Leaf nodes are our **base cases** – $dp(v) = 1$ if v is a leaf!
- The recursion for this DP is simply a DFS traversal of the tree. Hence our DP runs in $O(N + M) \sim O(N + N - 1) \sim O(N)$.

Directed Acyclic Graphs (DAGs)

There are, however, directed graphs that aren't trees but have no cycles.

Directed Acyclic Graphs

A **DAG** is a graph with **directed edges** such that you can never get from a node u to itself by following a sequence of edges. In other words, there are no cycles.

In general, our special DP graph is a DAG! Some properties for finite DAGs:

- There is always at least one 'source' vertex.
- There is always at least one 'sink' vertex – these are our **base cases**!

Like trees, if our underlying DP graph is a DAG, there's theoretically nothing stopping us from just doing DP on these graphs!

Delayed Execution

Delayed Execution

Your execution day has arrived, and you're headed for the execution building in your city. There are N buildings in your city, joined by M directional roads – the i 'th points from u_i to v_i , and takes w_i time to travel. Since traffic that loops onto itself is bad feng-shui, there's no way to get from a building back to itself by taking a series of roads.

You're allowed to choose your current location as well as the location of the execution. What's the longest possible time you can spend travelling to your demise, if you can't stop at any point?

Delayed Execution – Diagram

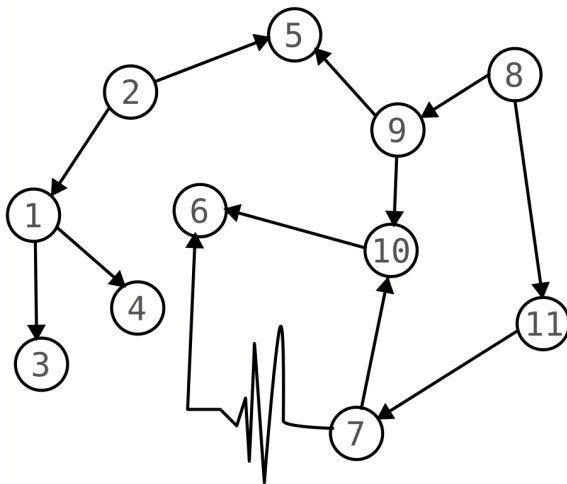


Figure: Funky city

Delayed Execution – Solution

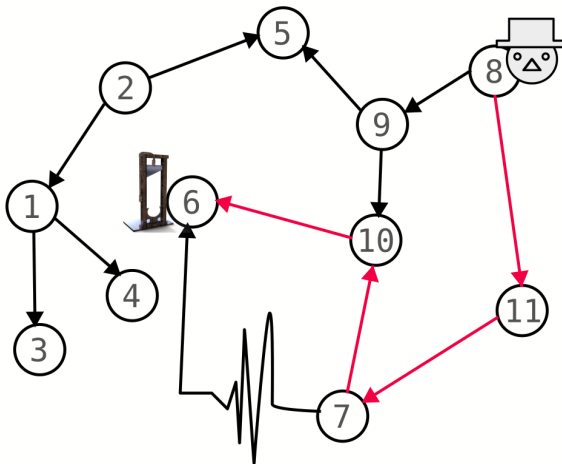


Figure: Rip bozo

Delayed Execution – Solution

We're trying to find the longest path on a DAG... we can solve this in $O((N + M) \log N)$?

- Looks a lot like Dijkstra, which solves for **shortest path**.
- Let's just map $w_i \mapsto -w_i$ for all edges i .
- There are no cycles on a DAG by definition, so we can just run Dijkstra without fear!

But spoiler – we can actually do this in linear time (i.e. $O(N + M)$); how?

- Let's define the **state** $dp(v) := \text{length of the longest path starting at } v$.
- We can compute this based on v 's successor (neighbour) nodes!
 - We can think of the longest path starting at v as moving to one of v 's neighbours, and then recursively taking the longest path starting from there!
 - So, our transition is $dp(v) = w_{v \rightarrow i} + \max_{i \in \text{adj}(v)} dp(i)$.
- Our **base cases** are simply sink vertices with no neighbours, where $dp(v) = 0$.
- We obtain the answer by finding $\max_v dp(v)$. As we just do DFS, it's $O(N + M)$!

Outline

1 Housekeeping

2 Revision of DP

- What is DP
- 1D DP
- Recursion vs Iteration
- 2D+ DP

3 Touring the classics

- The Knapsack Problem
- Longest Increasing Subsequence
- Longest Common Subsequence

4 On graphs??

- DP on trees
- DP on DAGs

5 Appendix and resources

Dynamic Programming – Resources

The **USACO Guide** has good resources (and problems) for all levels of dynamic programming:

<https://usaco.guide/gold/intro-dp?lang=cpp>

For a set of (increasing difficulty) educational DP problems, see AtCoder's Educational DP Contest:

<https://atcoder.jp/contests/dp/tasks>

And if you missed our first DP workshop, you can catch up on the slides here:

<https://unswcpmsoc.com/workshops/programming/2026/t1w8.pdf>

There's also a livestream on our YouTube channel!

Feedback!

Please fill out feedback form so we know how to run future workshops :D



(a) Feedback form



(b) Attendance form

Photo time :)

